

Penetration Test Report

Application: [TARGET]
Test Date: [DATA]
Report Author: [REDACTED]
Document Version: 1.0

Document Purpose

This report contains the results of penetration tests conducted on the web application [TARGET]. The purpose of the tests was to identify security vulnerabilities according to the OWASP Top 10 2025 methodology, assess their impact on application security, and provide remediation recommendations.

Test Scope: Web application

Methodology: Manual security testing, HTTP request analysis, JavaScript code analysis, injection testing, access control testing

1. Risk handling recommendation

Vulnerabilities	
Risk impact	Description
Critical	It is recommended to take immediate mitigating actions or limit the possibility of vulnerability exploitation.
High	It is recommended to take mitigating actions as soon as possible.
Medium	The mitigating actions should be taken after eliminating the vulnerabilities with critical and high risk impact.
Low	The mitigating actions should be taken after eliminating the vulnerabilities with critical, high, and medium risk impact.
Recommendations	
The decision whether to take mitigating actions should be made by the client.	

2. Vulnerabilities

2.1 SQL Injection in the ?[PARAM]= parameter

Risk impact	Critical	CVSS	9.8	ASVS	V5
Exploitation conditions	Requires being logged in as a user and the ability to send HTTP requests with the [PARAM] parameter or execute the JavaScript function [FUNC](). The parameter is available after authentication.				
Exploitation results	Ability to read database records via UNION SELECT. It was determined that the target query returns 2 columns, enabling further database exploration.				
References	OWASP Top 10 2025 - A05:2025 - Injection				
Remediation	1. Use parameterized queries (prepared statements) for the [PARAM] parameter. 2. Validate that [PARAM] is an integer - reject any other values. 3. Apply the principle of least privilege for the database user used by the application.				

Vulnerability description:

The [PARAM] parameter in HTTP requests is used to display details of a specific database record. Testing revealed that this parameter is vulnerable to SQL Injection attacks.

The vulnerability was confirmed in two ways:

1. Directly in HTTP requests:
 - a. The payload
POST /?[PARAM]=1%20UNION%20SELECT%20null,null-- was sent - the server returned a 200 status code.
 - b. Tests with a single null (UNION SELECT null--) returned a 500 error, while with double null (UNION SELECT null,null--) - returned 200. This indicates that the target SQL query returns 2 columns.
2. Via the JavaScript [FUNC]() function:
 - a. [FUNC]('dataDetails', '?[PARAM]=1 UNION SELECT null,null--') - does not return a 500 error; the page loads correctly, confirming that the

server processed the injected query.

- b. `[FUNC]('dataDetails', '?[PARAM]=1 UNION SELECT null--')` - returns an error, further confirming that the target query operates on 2 columns.

Both methods (HTTP and JavaScript function) yield consistent results, unequivocally confirming the existence of the SQL Injection vulnerability in the `?[PARAM]=` parameter.

Test case:

1. Log in to the application as a user.
2. Method 1 - HTTP request:
 - a. Send the following request:
`POST /[PARAM]=1%20UNION%20SELECT%20null,null-- HTTP/1.1`
 - b. The server responds with a 200 OK status code.
3. Method 2 - JavaScript function:
 - a. In the browser console (F12), execute:
`[FUNC]('dataDetails', '?[PARAM]=1 UNION SELECT null,null--')`
 - b. The page loads without a 500 error.
4. The test with `UNION SELECT null--` (single column) returns a 500 error in both HTTP and `[FUNC]()`, indicating that the original query operates on 2 columns.
5. This confirms that the server executed the injected SQL query - confirming the SQL Injection vulnerability.

2.2 Unauthorized access to the administration panel and login logs

Risk impact	Critical	CVSS	9.8	ASVS	V1
Exploitation conditions	Requires being logged in as a user (e.g. <code>[USER]</code>) and the ability to execute the JavaScript function <code>[FUNC]()</code> in the browser console.				
Exploitation results	A user with <code>[USER]</code> privileges can gain full access to the administration panel containing login logs. These logs contain login attempt statuses, enabling user enumeration and brute-force attacks.				
References	OWASP Top 10 2025 - A01:2025 - Broken Access				

	Control
Remediation	1. Implement server-side authorization for every navigation function call regardless of the task parameter. 2. Verify user permissions before displaying the administration panel and login logs. 3. Log statuses should not disclose information that allows distinguishing between existing and non-existing users.

Vulnerability description:

The application has an administration panel with login history that should only be accessible to users with administrator privileges. The navigation mechanism in the application is based on the task parameter passed to the JavaScript function `[FUNC]()`. Tests showed that the server does not verify user permissions before executing the requested action.

The vulnerability was confirmed as follows:

- While logged in as a `[USER]` user, the following was executed in the browser console:
`[FUNC]('admin', '')`
- As a result, the `[USER]` user gained access to the administration panel with login logs.

The panel displays the full login history containing: usernames, login attempt dates and times, login attempt statuses(0 - correct login and password, 1 - correct login, incorrect password, 2 - incorrect login and password)

These statuses enable an attacker to enumerate existing users and conduct brute-force attacks on passwords. This is particularly dangerous when combined with other application vulnerabilities, such as the lack of account lockout and weak authentication mechanisms.

Test case:

1. Log in to the application as a `[USER]` user.
2. Open the browser console (F12).
3. In the console, execute:
`[FUNC]('admin', '')`
4. Observe that the application interface displays the administration panel with login logs, which should not be accessible to the `[USER]` user.

5. This indicates a complete lack of server-side access control - the [USER] user can perform actions reserved for the administrator.

2.3 Lack of SSL/TLS - sensitive data, including credentials and session tokens, is transmitted in plaintext

Risk impact	High	CVSS	7.5	ASVS	V9
Exploitation conditions	Requires the ability to intercept network traffic between the user and the server (e.g. on an unsecured Wi-Fi network, as a Man-in-the-Middle).				
Exploitation results	An attacker can intercept login credentials (MD5 hash), session tokens (SessionState), and other sensitive data transmitted between the browser and the server. This enables session hijacking and unauthorized access to user accounts without knowing the password.				
References	OWASP Top 10 2025 - A04:2025 - Cryptographic Failures				
Remediation	1. Deploy an SSL/TLS certificate for the domain [TARGET]. 2. Configure redirection from HTTP to HTTPS. 3. Implement the HSTS (Strict-Transport-Security) header to enforce HTTPS usage.				

Vulnerability description:

The application is only available via HTTP. Attempting to connect via HTTPS (https://[TARGET]/) fails, meaning all traffic between the user's browser and the server is transmitted in plaintext.

This means that:

- login credentials (MD5 hash sent in the login request)
- session token (SessionState)
- all transmitted data (including logs, messages, filter parameters)

can be intercepted by an attacker performing a Man-in-the-Middle (MITM) attack on an unsecured network, such as public Wi-Fi.

The lack of SSL/TLS, combined with other vulnerabilities (e.g. lack of HttpOnly and Secure flags for cookies, weak authentication mechanisms), significantly increases the risk of session hijacking and account compromise.

Test case:

1. The page `http://[TARGET]/` was opened in the browser.
2. An attempt to access `https://[TARGET]/` failed, the page is unavailable.
3. Using a traffic interception tool (e.g. Burp Suite), a login request was captured.
4. The captured request shows all data in plaintext, including:
 - a. login
 - b. MD5 password hash
 - c. SessionState
5. This means that any attacker on the same network can intercept login credentials and session tokens.

2.4 Weak client-side authentication mechanism using MD5 and an ignored salt

Risk impact	High	CVSS	8.1	ASVS	V2
Exploitation conditions	Requires the ability to intercept network traffic (e.g. MITM) or access to HTTP request logs to obtain the MD5 hash.				
Exploitation results	An attacker can intercept the MD5 hash of the password and reuse it to log into the user's account without knowing the original password (Pass-the-Hash). The server ignores the salt value, further facilitating the attack.				
References	OWASP Top 10 2025 - A07:2025 - Authentication Failures				
Remediation	1. Password hashing should be performed exclusively on the server side using a strong algorithm (e.g. bcrypt, Argon2, PBKDF2). 2. Eliminate client-side hashing - transmitting the password in plaintext over HTTPS is more secure than sending an MD5 hash. 3. Implement a proper salt mechanism - salt should be generated and stored server-side, not in the browser.				

	4. The server should not accept intercepted hashes as valid passwords.
--	--

Vulnerability description:

The application implements an authentication mechanism where the password is hashed client-side using the MD5 algorithm before being sent to the server. The salt is generated by the `Math.random()` function in the browser and included in the request.

Tests revealed the following issues:

1. Client-side hashing: the password is hashed in the browser using MD5. An intercepted hash can be used to log in without knowing the original password.
2. Server ignores salt: during testing, the salt value in the login request was changed to an arbitrary value, and the server still authenticated the user. This means the server does not verify or use the provided salt - it only compares the password hash located in the middle of the request.
3. Pass-the-Hash: the intercepted login request has the format:
password=<hash(salt)>|<hash(password)>|<hash(salt)>

An attacker can intercept this request and replay it, gaining access to the user account without knowing the password. Additionally, since the server ignores the salt, the attacker can change the first and third parts (salt hashes) to any value and the request will still be accepted.

4. Weak algorithm MD5 is vulnerable to collision attacks and can be cracked using brute-force or rainbow tables, especially for weak passwords.

Test case:

1. Log in to the application while intercepting the HTTP request using a tool (e.g. Burp Suite).
2. The intercepted request contains parameters in the format:
login=[USER]&password=<hash(salt)>|<hash(password)>|<hash(salt)>
3. Change the first and third parts (salt hashes) to any arbitrary values:
login=[USER]&password=arbitrary_hash|<hash(password)>|arbitrary_hash
4. Send the modified request - the server accepts it and logs the user in.
5. This means the server ignores the salt value and accepts the intercepted password hash as valid authentication.

2.5 Reflected XSS in the ?[PARAM2]= parameter, with potential SessionState token exposure

Risk impact	High	CVSS	8.1	ASVS	V5
Exploitation conditions	Requires being logged in as a user and the ability to execute the JavaScript function [FUNC]() in the browser console or tricking the victim into clicking a crafted link.				
Exploitation results	An attacker can execute arbitrary JavaScript code in the victim's browser. Since the application does not use cookies or Session Storage, and the session relies on the SessionState token placed in the DOM without HttpOnly protection, the attacker can read the SessionState token and hijack the user's session.				
References	OWASP Top 10 2025 - A05:2025 - Injection				
Remediation	1. Encode/validate input in the [PARAM2] parameter before reflecting it in the response (e.g. HTML entity encoding). 2. Implement Content-Security-Policy (CSP) to limit the ability to execute untrusted code. 3. Store the SessionState token in a cookie with the HttpOnly flag to prevent JavaScript from reading it.				

Vulnerability description:

The [PARAM2] parameter in requests to the administration panel is used for filtering login logs by date. Tests showed that this parameter is vulnerable to Reflected XSS attacks.

The vulnerability was confirmed using the JavaScript [FUNC]() function:

- While logged in as a [USER] user, the following was executed in the browser console:
`[FUNC]('admin', '?[PARAM2]='"><img src=x onerror=alert(1)>')`
- As a result, the JavaScript code was executed in the browser, displaying an alert(1) window.

Since the application does not use cookies or Session Storage, and the session relies on the SessionState token placed in the DOM (accessible to JavaScript), exploiting XSS could allow an attacker to:

- Read the SessionState token from the DOM

- Send it to the attacker's server
- Hijack the user's session (Session Hijacking)

Test case:

1. Log in to the application as any user.
2. Open the browser console (F12) and execute:
[FUNC]('admin', '?[PARAM2]='">')
3. Observe that an alert(1) window appears in the browser.
4. This means the [PARAM2] parameter is vulnerable to Reflected XSS.

2.6 IDOR in the ?[PARAM]= parameter allowing access to unauthorized messages

Risk impact	High	CVSS	7.5	ASVS	V1
Exploitation conditions	Requires being logged in as a user (e.g. [USER]) and the ability to modify the [PARAM] parameter in the request or execute the [FUNC]() function with an appropriate value.				
Exploitation results	A user with [USER] privileges can gain access to messages of other users that they should not have access to.				
References	OWASP Top 10 2025 - A01:2025 - Broken Access Control				
Remediation	1. Verify user permissions on the server before returning data for the [PARAM] parameter. 2. Associate records with a specific user and check this association with every query. 3. Implement authorization mechanisms, not just authentication.				

Vulnerability description:

The [PARAM] parameter in HTTP requests is used to display details of a specific record (e.g. a message) from the database. Tests showed that the server does not verify whether the logged-in user has permission to read the given record.

The vulnerability was confirmed as follows:

- While logged in as a [USER] user, the following was executed in the browser console:

```
[FUNC]('dataDetails', '?[PARAM]=2')
```

As a result, the [USER] user gained access to a message that should not be available to them.

- Similarly, by sending a direct HTTP request:

```
POST /[PARAM]=2 HTTP/1.1
```

The server returned the record data that the [USER] user should not have access to.

Tests also showed that changing the [PARAM] parameter manually in the URL causes a logout, while using the [FUNC]() function allows bypassing this protection and gaining access to unauthorized data.

Test case:

1. Log in to the application as a [USER] user.
2. Open the browser console (F12) and execute:
[FUNC]('dataDetails', '?[PARAM]=2')
3. Observe that message details are displayed that the [USER] user should not have access to.
4. Alternatively, send an HTTP request:
POST /[PARAM]=2 HTTP/1.1
5. The server returns the record data, confirming the lack of access control.

2.7 Missing account lockout after failed login attempts

Risk impact	Medium	CVSS	5.3	ASVS	V2
Exploitation conditions	Requires the ability to send multiple login requests. Usernames can be obtained through the IDOR vulnerability in login logs (described in section 2.2) or enumeration based on response time (described in section 2.8), which facilitates the attack.				
Exploitation results	An attacker can conduct brute-force attacks on passwords, as the server does not lock accounts or IP addresses after failed login attempts, allowing an unlimited number of requests to be sent. Combined with				

	other vulnerabilities (e.g. IDOR to logs, user enumeration), it enables effective account compromise.
References	OWASP Top 10 2025 - A07:2025 - Authentication Failures
Remediation	1. Implement account lockout after a defined number of failed login attempts (e.g. 5 attempts). 2. Apply rate limiting for login requests. 3. Implement CAPTCHA after several failed attempts. 4. Log failed attempts and monitor suspicious activity.

Vulnerability description:

The application does not implement any account lockout mechanism or login attempt restrictions. Tests showed that the server allows an unlimited number of login requests for the same account without any consequences. The server does not lock accounts or IP addresses.

The vulnerability was confirmed as follows:

- Multiple login requests with an incorrect password were sent for the same user.
- The server did not lock the account or IP address after many failed attempts.
- No protective mechanisms such as CAPTCHA or attempt limits were present.

Potential consequences:

The lack of account and IP lockout enables brute-force attacks on passwords for existing accounts, combined with:

- IDOR to login logs (section 2.2) - allows enumeration of existing users
- Username enumeration based on response time (section 2.8)
- Weak authentication mechanisms (MD5, Pass-the-Hash)
- Lack of two-factor authentication

it enables an attacker to effectively compromise user accounts.

Test case:

1. Send multiple login requests with an incorrect password for the same account.
2. Observe that the server does not lock the account or IP address after many failed attempts.
3. The brute-force attack on passwords can be continued indefinitely.

2.8 Username enumeration through response-time differences

Risk impact	Medium	CVSS	5.3	ASVS	V2
Exploitation conditions	Requires the ability to send multiple login requests and measure server response times.				
Exploitation results	An attacker can determine which usernames exist in the system based on differences in server response times. This enables targeted brute-force attacks on passwords for existing accounts.				
References	OWASP Top 10 2025 - A07:2025 - Authentication Failures				
Remediation	1. Implement constant response time for all login attempts, regardless of whether the user exists. 2. Apply rate limiting for login requests. 3. Return identical error messages for all failed login attempts.				

Vulnerability description:

The application allows determining whether a username exists in the system based on differences in server response times. Tests showed that the response time for an existing user is significantly longer than for a non-existing user.

The vulnerability was confirmed as follows:

- Login requests were sent with an existing username (e.g. [USER]) and with a non-existing username.
- A significant difference in response time was observed for an existing user, the response time was longer.
- This difference allows an attacker to enumerate existing users in the system.

Username enumeration enables an attacker to:

- Determine which accounts exist in the system
- Conduct targeted brute-force attacks on passwords for existing accounts
- Combined with the lack of account lockout (section 2.7) and IDOR to logs (section 2.2), it significantly increases the risk of account compromise

Test case:

1. Send a login request with a non-existing username and measure the response time.
2. Send a login request with an existing username (e.g. [USER]) and measure the response time.
3. Compare the response times - for the existing user, the time is significantly longer.
4. This means that based on response time, it is possible to determine which usernames exist in the system.

2.9 Publicly accessible /[FILE].php and /[FILE]

Risk impact	Medium	CVSS	5.3	ASVS	V5
Exploitation conditions	Requires only access to a web browser and knowledge of the /[FILE].php or /[FILE] paths.				
Exploitation results	An attacker can obtain detailed information about the server configuration, including PHP version, installed extensions, system paths, environment variables, and other sensitive configuration data. This information can be used to prepare targeted attacks on the server.				
References	OWASP Top 10 2025 - A02:2025 - Security Misconfiguration				
Remediation	1. Remove the /[FILE].php and /[FILE] files from the production environment. 2. Configure the web server to block access to these resources (e.g., via Apache .htaccess rules or NGINX configuration). 3. Implement a policy to minimize information disclosed about the server.				

Vulnerability description:

Tests showed that the /[FILE].php and /[FILE] files are publicly accessible on the application server. The phpinfo() function displays detailed information about the PHP configuration, which should not be available in the production environment.

The vulnerability was confirmed as follows:

- A GET request was sent to http://[TARGET]/[FILE].php.

- The server returned a response with status code 200, displaying the full PHP configuration.
- Similarly, a request to `http://[TARGET]/[FILE]` also returned status code 200.

Information disclosed includes, among others:

- PHP version
- installed extensions
- system paths
- environment variables
- web server configuration

can be used by an attacker to:

- Identify outdated or vulnerable versions of PHP and extensions
- Understand the file system structure
- Prepare targeted attacks on the server
- Explore potential configuration weaknesses

Test case:

1. Open the following address in a browser: `http://[TARGET]/[FILE].php` or `http://[TARGET]/[FILE]`
2. Observe that a page with full PHP configuration is displayed.

2.10 Missing CSRF protection

Risk impact	Medium	CVSS	5.3	ASVS	V4
Exploitation conditions	Requires the user to be logged in and to be tricked into visiting a malicious website or clicking a crafted link.				
Exploitation results	An attacker can send unauthorized requests on behalf of a logged-in user, using their session. This could lead to performing unwanted actions, such as accessing the administration panel, reading logs, or other operations available to that user.				
References	OWASP Top 10 2025 - A01:2025 - Broken Access Control				
Remediation	1. Implement CSRF tokens (e.g. synchronizer token pattern) for all forms and requests that modify the application state. 2. Verify the CSRF token server-side for each request.				

	3. Use the SameSite attribute for cookies.
--	--

Vulnerability description:

The forms in the application do not contain any mechanisms protecting against CSRF (Cross-Site Request Forgery) attacks. There is no CSRF token field or other protections that would prevent unauthorized requests from being sent from an external site.

The vulnerability was confirmed by analyzing forms in the application, including:

- login form
- navigation form
- data filtering form

None of the forms contained a hidden field with a CSRF token or any other protection mechanism, no security headers protecting against CSRF were present.

The lack of CSRF protection enables an attacker to:

- Send a [FUNC]('admin', ") request on behalf of a logged-in user, which (combined with the vulnerability described in section 2.2) allows unauthorized access to the administration panel.
- Perform other actions available to that user without their knowledge.
- Exploit the user's session to perform unwanted operations.

Test case:

1. Log in to the application as a user.
2. Analyze the source code of the forms in the application.
3. Observe that none of the forms contains a CSRF token field.
4. This means the application is vulnerable to CSRF attacks.

2.11 Use of SHA-1 in the SessionState mechanism

Risk impact	Medium	CVSS	5.3	ASVS	V3
Exploitation conditions	Requires the ability to intercept the SessionState token (e.g. through MITM, XSS, or IDOR attacks) and knowledge of the algorithm used to generate it.				
Exploitation results	An attacker can perform a collision attack on the SHA-1 algorithm, which in the future could enable forging a session token and hijacking another user's account.				
References	OWASP Top 10 2025 - A04:2025 - Cryptographic				

	Failures
Remediation	1. Use stronger cryptographic algorithms for generating session tokens, e.g. SHA-256 or SHA-3. 2. Generate session tokens with appropriate length (minimum 128 bits of entropy). 3. Session tokens should be stored in cookies with the HttpOnly and Secure flags.

Vulnerability description:

Analysis of the session mechanism showed that the SessionState token is generated using the obsolete SHA-1 algorithm. The token consists of compressed binary data with an SHA-1 hash, rather than a secure, random session identifier.

The algorithm was identified based on the hash length - SHA-1 generates a 160-bit hash, which is represented as 40 hexadecimal characters. The SessionState token intercepted from HTTP requests contained exactly a 40-character string, indicating the use of SHA-1. For comparison: MD5 generates 32 characters, while SHA-256 generates 64 characters.

SHA-1 is a cryptographic algorithm considered broken, enabling collision attacks (e.g., SHAttered). An attacker can generate two different session tokens with the same hash, potentially allowing them to forge another user's token and hijack their session. Combined with other vulnerabilities (e.g., lack of SSL/TLS, XSS, IDOR), the risk of session hijacking increases significantly.

Test case:

1. Log in to the application and intercept an HTTP request containing the SessionState token.
2. Analyze the structure of the SessionState token.
3. Identify that the token contains a 40-character hexadecimal hash, indicating the SHA-1 algorithm, and compressed binary data.
4. This means the session mechanism uses an outdated and potentially unsafe algorithm.

2.12 Missing security headers: CSP, X-Frame-Options and X-Content-Type-Options

Risk impact	Low	CVSS	3.7	ASVS	V14
Exploitation conditions	Requires finding another vulnerability (e.g. XSS) to exploit it or tricking the victim into interacting with the application in a specific way.				
Exploitation results	Lack of CSP enables execution of injected JavaScript code in the event of XSS. Lack of X-Frame-Options allows Clickjacking attacks. Lack of X-Content-Type-Options enables MIME-sniffing attacks.				
References	OWASP Top 10 2025 - A02:2025 - Security Misconfiguration				
Remediation	1. Implement the Content-Security-Policy (CSP) header with appropriate directives to restrict resource sources. 2. Add the X-Frame-Options: DENY or SAMEORIGIN header to protect against Clickjacking. 3. Add the X-Content-Type-Options: nosniff header to protect against MIME-sniffing.				

Vulnerability description:

Analysis of HTTP response headers revealed the absence of key security headers that should be configured by default in every web application.

Missing headers:

- **Content-Security-Policy (CSP)** - allows restricting the sources from which resources (scripts, styles, images) can be loaded, preventing unauthorized code execution in the event of XSS.
- **X-Frame-Options** - prevents the application from being embedded in iframes on other sites, protecting against Clickjacking attacks.
- **X-Content-Type-Options: nosniff** - prevents the browser from MIME type sniffing, protecting against attacks that rely on files being interpreted as a different type than declared.

The absence of these headers increases the risk of exploiting other vulnerabilities, such as XSS, Clickjacking, or MIME-sniffing.

Test case:

1. Log in to the application and intercept an HTTP response using a tool (e.g. Burp Suite).

2. Analyze the response headers.
3. Observe that the following headers are missing:
 - a. Content-Security-Policy
 - b. X-Frame-Options
 - c. X-Content-Type-Options
4. This means the application is vulnerable to attacks exploiting the lack of these protections.

2.13 Server version disclosure through HTTP headers

Risk impact	Low	CVSS	3.7	ASVS	V14
Exploitation conditions	Requires only sending an HTTP request to the server and reading the response headers.				
Exploitation results	An attacker can obtain information about the web server (Apache) and PHP versions, enabling identification of known vulnerabilities for those versions and preparation of targeted attacks.				
References	OWASP Top 10 2025 - A02:2025 - Security Misconfiguration				
Remediation	1. Disable or hide server version information in HTTP headers (e.g., in Apache by setting ServerTokens Prod and ServerSignature Off). 2. Hide the X-Powered-By header (e.g., in PHP by setting expose_php = Off). 3. Implement a policy to minimize information disclosed about the server.				

Vulnerability description:

During the analysis of HTTP response headers, it was observed that the server returns detailed information about its software. The following headers are visible in the responses:

- Server: Apache/[VERSION] - revealing the Apache web server version
- X-Powered-By: PHP/[VERSION] - revealing the PHP version used on the server

Disclosing this information does not pose a direct threat, but it significantly facilitates further attacker actions. Knowing the exact software versions, an attacker can check

which known vulnerabilities affect these versions and prepare precisely targeted attacks. Additionally, these are highly outdated versions - Apache [VERSION] has not been supported since 2017, and PHP [VERSION] since 2014, meaning that many vulnerabilities present in them have never been patched.

Test case:

1. A GET request was sent to http://[TARGET]/.
2. The response headers were analyzed.
3. The presence of Server and X-Powered-By headers revealing software versions was observed.
4. This means the server discloses detailed configuration information.

Summary

The application is in a very poor security state. The penetration tests revealed 13 vulnerabilities, including two critical ones that allow an attacker to gain full control over the data and functionality of the application. The application does not meet basic security standards, it uses outdated, unsupported software (Apache [VERSION], PHP [VERSION]), does not encrypt traffic, and key mechanisms such as authorization and input validation are completely omitted or implemented incorrectly.

Particularly concerning is that many vulnerabilities can be combined, creating effective attack chains:

1. SQL Injection + IDOR + lack of authorization - an attacker can read any data from the database, including logs, passwords, and session tokens.
2. IDOR to logs + lack of account lockout - an attacker can enumerate users based on log statuses, and then conduct brute-force attacks on passwords without the risk of lockout.
3. Lack of SSL/TLS + weak client-side authentication (MD5) - an attacker on an unsecured network can intercept the MD5 hash and reuse it for login (Pass-the-Hash).
4. Lack of authorization + lack of CSRF - an attacker can trick a logged-in user into performing unauthorized actions, such as accessing the administration panel.
5. XSS + lack of CSP - an attacker can inject JavaScript code, read the SessionState token from the DOM, and hijack the user's session.

The application requires immediate intervention, implementing fixes for critical and high-risk vulnerabilities, as well as a thorough modernization of the technology stack and implementation of basic security mechanisms that are standard in modern web applications.

Test Process Overview

Planning

The testing process began with reconnaissance of the application, during which the functionality, site structure, and technologies used on both the server and client sides were analyzed. All available resources, URL parameters, forms, and JavaScript functions were identified. Based on the collected information and OWASP Top 10 2025 guidelines, a detailed test checklist was developed, covering access control, injection, authentication, and configuration tests.

Execution

The tests were conducted manually, using Burp Suite for intercepting and modifying HTTP requests, browser developer tools (DevTools) for JavaScript code analysis, the browser console for invoking functions, and HTTP header analysis tools.

The testing process covered a wide range of scenarios: access control tests (verification of permissions at [USER LEVEL 1], [USER LEVEL 2], and [ADMIN] levels, IDOR, privilege escalation), injection tests (SQL Injection, XSS, path traversal), authentication tests (analysis of the login mechanism, pass-the-hash, brute-force, user enumeration), configuration tests (security headers, configuration files, HTTP methods), session management tests (SessionState analysis, token replacement), and business logic tests (enforcement of permissions independently of the interface).

Reporting

The test results were compiled in the form of a detailed report containing a description of each identified vulnerability along with its OWASP Top 10 2025 classification, risk assessment, exploitation conditions, potential consequences, and remediation recommendations. Each vulnerability was supported by a test case enabling its reproduction. The report concludes with a summary of the overall application security state and identification of the most important remediation priorities.

[REDACTED]
[DATA]
[REDACTED]